

APT-DTS: Active Prompt Tuning with Density-Tree (Steepest-Uphill) Sampling

Abhiram Kandiyana

Context. This document extends Active Prompt Tuning (APT) by replacing the *random draw* of the active set with a *Density-Tree* Sampling (DTS) rule computed in the VLM embedding space. The density-tree (steepest-uphill) construction used in APT-DTS is inspired by *density-based clustering via uphill dynamics* in [2], Chapter 6 (*Density estimation in action*), Section 6.2 (*Cluster analysis*), especially Subsection 6.2.1 and Subsection 6.2.2.

1 Symbol Glossary

Symbol	Meaning
$\mathcal{X}, \mathcal{Y}$	image space and class set $\{1, \dots, K\}$
$\mathcal{D}_\ell, \mathcal{D}_u$	labeled set, unlabeled pool
S	system prompt
$g_\theta(\cdot)$	VLM query function returning $(\hat{y}, \hat{e})$
$\mathcal{P}^{(r)}$	prompt set after round r
$A^{(r)}$	active set in round r
$n_{\min}$	per-class minimum in $A^{(r)}$ (APT coverage guard; uses $\hat{y}$)
$\phi(\cdot)$	image embedding function (VLM encoder)
$z_i \in \mathbb{R}^d$	embedding of unlabeled sample x_i
$d(\cdot, \cdot)$	distance in embedding space (e.g., cosine)
$\mathcal{N}_k(i)$	k nearest neighbors of point i
k	kNN graph size
k_ρ	number of neighbors used for density proxy
ρ_i	local density proxy at point i
k_t	neighbor index defining threshold radius t_i
t_i	threshold radius limiting candidate parents
$\pi(i)$	parent pointer of node i (steepest-uphill)
$r(i)$	root reached by following $\pi(\cdot)$
B_u	cluster corresponding to root u ; $ B_u $ is its size
k_b	neighborhood size used to compute boundary score
$b(i)$	boundary score (fraction of neighbors in other clusters)
$(\tau, s), R_{\max}$	APT stopping thresholds; max rounds

2 Problem Setup (from APT)

Let $\mathcal{X}$ denote the image space and $\mathcal{Y} = \{1, \dots, K\}$ the set of class indices. We are given: (i) a labeled set $\mathcal{D}_\ell = \{(x_i, y_i)\}_{i=1}^L \subset \mathcal{X} \times \mathcal{Y}$, (ii) an unlabeled pool $\mathcal{D}_u = \{x_j\}_{j=1}^U \subset \mathcal{X}$, (iii) a vision-language model (VLM) f_θ queried via a function

$$(\hat{y}, \hat{e}) = g_\theta(x; S, \mathcal{P}^{(r-1)}), \quad (1)$$

where S is a fixed system prompt, $\mathcal{P}^{(r-1)}$ is the previous prompt set, $\hat{y} \in \mathcal{Y}$ is the predicted class (parsed from structured output), and $\hat{e}$ is a brief explanation.

APT iteratively builds a prompt set of image-caption pairs; an oracle verifies labels and minimally edits explanations into corrected captions; corrected pairs are appended to the prompt set.

3 Key Additional Assumption for APT-DTS

Although the VLM may be closed-source and logit-free, we assume access to an image embedding function

$$\phi : \mathcal{X} \rightarrow \mathbb{R}^d, \quad z = \phi(x), \quad (2)$$

returning a fixed-dimensional embedding z for an image x (e.g., the VLM image encoder output).

4 APT-DTS: Density-Tree Sampling in Embedding Space

4.1 High-level Hypothesis (motivation)

In a semantically meaningful embedding space, unlabeled samples form clusters (*basins of attraction*) around local density *modes*. Decision-relevant ambiguity often concentrates near *valleys/saddles* separating these clusters. The density-tree (steepest-uphill) construction approximates these clusters using only local neighborhood comparisons; selecting samples near cluster boundaries yields informative supervision without requiring class knowledge.

4.2 Step 0: Precompute embeddings (per round or once)

For each $x_j \in \mathcal{D}_u$, compute $z_j = \phi(x_j)$. If ϕ is fixed across rounds, embeddings can be computed once and updated by removing selected indices.

4.3 Step 1: Build a local kNN graph

For each point z_i , find its k nearest neighbors in $\{z_j\}$ under a chosen distance $d(\cdot, \cdot)$ (typically cosine distance or Euclidean distance on normalized embeddings). Let $\mathcal{N}_k(i)$ denote the set of kNN indices of i .

Optional robustness (recommended): mutual kNN. Define the *mutual* neighborhood

$$\mathcal{N}_k^{\text{mut}}(i) = \{j \in \mathcal{N}_k(i) : i \in \mathcal{N}_k(j)\}.$$

Using mutual kNN reduces spurious edges and helps prevent “collective outlier islands” [1] from dominating boundary selection.

4.4 Step 2: Estimate local density (rank-based)

We use a simple kNN density proxy (any monotone proxy is acceptable; only local ordering matters). Let k_ρ (typically $k_\rho \leq k$) be the density neighborhood size. Define

$$\rho_i = \left(\frac{1}{k_\rho} \sum_{j \in \mathcal{N}_{k_\rho}(i)} d(z_i, z_j) \right)^{-1}. \quad (3)$$

Higher ρ_i indicates more locally dense (more “central/typical”) regions.

4.5 Step 3: Threshold radius t_i (locality constraint)

We define a per-point threshold radius t_i that limits which neighbors can be considered for uphill linking:

$$t_i = d(z_i, z_{i, (k_t)}), \quad (4)$$

where $z_{i, (k_t)}$ is the k_t -th nearest neighbor of i (with $k_t \leq k$). Equivalently, one may set t_i to a fixed percentile of kNN distances. This enforces that uphill transitions remain *local*.

4.6 Step 4: Steepest-uphill parent pointers (the density tree)

For each point i , define candidate parents:

$$\mathcal{C}(i) = \{j \in \mathcal{N}(i) : d(z_i, z_j) \leq t_i \wedge \rho_j > \rho_i\}, \quad (5)$$

where $\mathcal{N}(i)$ is either $\mathcal{N}_k(i)$ or $\mathcal{N}_k^{\text{mut}}(i)$.

If $\mathcal{C}(i) = \emptyset$, i is a *root*, i.e., a local density *mode* (a local maximum of ρ within its neighborhood).

Otherwise, set the parent $\pi(i)$ to the *steepest uphill* neighbor:

$$\pi(i) = \arg \max_{j \in \mathcal{C}(i)} \frac{\rho_j - \rho_i}{d(z_i, z_j)}. \quad (6)$$

This builds a directed forest (each node has at most one outgoing edge to its parent).

4.7 Step 5: Cluster assignment via root tracing

Define the root mapping $r(i)$ by following parent pointers until reaching a root:

$$r(i) = \begin{cases} i, & \text{if } \pi(i) \text{ is undefined (root)} \\ r(\pi(i)), & \text{otherwise.} \end{cases}$$

All points sharing the same $r(i)$ belong to the same *cluster*.

Let cluster sizes be

$$|B_u| = |\{i : r(i) = u\}|.$$

4.8 Step 6: Boundary score (cluster-separating regions)

To prioritize points near *valleys* between clusters, define a boundary score:

$$b(i) = 1 - \frac{1}{k_b} \sum_{j \in \mathcal{N}_{k_b}(i)} \mathbf{1}[r(j) = r(i)], \quad (7)$$

where k_b is a small neighborhood size for boundary scoring (e.g., $k_b \in [10, 50]$). Interpretation:

- If i lies deep inside a cluster, most neighbors share its root $\Rightarrow b(i) \approx 0$.
- If i lies near a boundary between clusters, neighbors split across roots $\Rightarrow b(i)$ increases.

4.9 Step 7: Collective-outlier safeguards (optional but recommended)

Collective outliers can form small, isolated clusters [1]. To avoid over-selecting them, apply:

- **Minimum cluster mass:** ignore (or cap) clusters with $|B_u| < m_{\min}^{\text{cluster}}$ when selecting boundary points.
- **Cap from tiny clusters:** allow at most c_{tiny} selections per tiny cluster per round.

These safeguards preserve the main benefit of boundary sampling while preventing label budget waste on isolated islands.

4.10 Step 8: Active set selection rule for APT

In round r , instead of random sampling, form the initial active set $A^{(r)}$ by selecting the top m points from $\mathcal{D}_u$ ranked by $b(i)$, subject to cluster safeguards. Optionally, you may mix in a small fraction of *mode representatives* (e.g., highest-density points in each large cluster) to maintain coverage of major modes; but the simplest variant is boundary-only ranking.

5 Round Structure (APT with DTS Sampling)

We keep the APT round structure unchanged *except* for step (i), which draws the initial active set using the DTS density-tree boundary score. The class-minimum constraint and oracle editing remain as in APT.

6 Algorithm: APT-DTS (next page)

7 Interpretation and Practical Notes

What DTS is doing. The parent-pointer forest approximates “flow” to local density maxima using only local comparisons. Clusters are induced by roots; boundary score $b(i)$ quantifies how often a point’s neighbors belong to other clusters, approximating valley/saddle regions between clusters.

Why this remains logit-free and closed-model compatible. APT-DTS requires no gradients, logits, or probabilities. It uses: (i) embeddings $\phi(x)$, (ii) distances and kNN, and (iii) the original APT VLM query g_θ for label/explanation generation and coverage guard.

Recommended default hyperparameters.

$$k \in [50, 200], \quad k_\rho \in [20, 80], \quad k_t \in [20, 80], \quad k_b \in [10, 50].$$

Cosine distance on ℓ_2 -normalized embeddings can be used. For outlier guards, setting $m_{\min}^{\text{cluster}}$ to a small fraction of the pool (e.g., 0.5%) or a fixed minimum (e.g., 20), and $c_{\text{tiny}} = 1$ is a good starting point.

Algorithm 1 APT-DTS: Active Prompt Tuning with Density-Tree Boundary Sampling

Require: Labeled set $\mathcal{D}_\ell$, unlabeled pool $\mathcal{D}_u$; system prompt S ; VLM query g_θ ; embedding function ϕ ; initial active-batch size m ; per-class minimum $n_{\min}$; thresholds (τ, s) ; max rounds $R_{\max}$.

Require: kNN params: k (graph), k_ρ (density), k_t (threshold), k_b (boundary).

Require: Optional outlier guards: mutual-kNN flag; $m_{\min}^{\text{cluster}}$, c_{tiny} .

Ensure: Final prompt set $\mathcal{P}^{(R)}$.

```
1: Seed construction (APT): stratified sample from  $\mathcal{D}_\ell$ ; expert writes captions; set  $\mathcal{P}^{(0)}$ .
2: for  $r = 1$  to  $R_{\max}$  do
3:   (APT-DTS selection) Build embeddings: for each  $x_i \in \mathcal{D}_u$ , compute  $z_i \leftarrow \phi(x_i)$ .
4:   Build kNN graph: compute  $\mathcal{N}_k(i)$  for all  $i$  (optionally mutual-kNN).
5:   Density proxy:  $\rho_i \leftarrow \left( \frac{1}{k_\rho} \sum_{j \in \mathcal{N}_{k_\rho}(i)} d(z_i, z_j) \right)^{-1}$ .
6:   Threshold radius:  $t_i \leftarrow d(z_i, z_{i, (k_t)})$ .
7:   Parent pointers:
8:   for  $i = 1$  to  $|\mathcal{D}_u|$  do
9:      $\mathcal{C}(i) \leftarrow \{j \in \mathcal{N}(i) : d(z_i, z_j) \leq t_i \wedge \rho_j > \rho_i\}$ .
10:    if  $\mathcal{C}(i) = \emptyset$  then
11:       $\pi(i) \leftarrow \text{NIL}$ .
12:    else
13:       $\pi(i) \leftarrow \arg \max_{j \in \mathcal{C}(i)} \frac{\rho_j - \rho_i}{d(z_i, z_j)}$ .
14:    end if
15:  end for
16:  Root assignment: for each  $i$ , set  $r(i) \leftarrow$  root obtained by following  $\pi(\cdot)$  until NIL.
17:  Boundary score:  $b(i) \leftarrow 1 - \frac{1}{k_b} \sum_{j \in \mathcal{N}_{k_b}(i)} \mathbf{1}[r(j) = r(i)]$ .
18:  Initial active set  $A^{(r)}$ :
19:    rank all  $i$  by  $b(i)$  descending; select top  $m$  subject to optional cluster guards.
20:     $A^{(r)} \leftarrow$  corresponding images from  $\mathcal{D}_u$ .
21:  VLM querying: for each  $x \in A^{(r)}$ , query  $g_\theta(x; S, \mathcal{P}^{(r-1)}) \rightarrow (\hat{y}, \hat{e})$ .
22:  Class-minimum constraint (APT coverage guard):
23:  while  $\exists k : n_k(A^{(r)}) < n_{\min}$  do
24:    sample  $x \sim \mathcal{D}_u \setminus A^{(r)}$ .
25:    query  $g_\theta(x; S, \mathcal{P}^{(r-1)}) \rightarrow (\hat{y}, \hat{e})$ .
26:     $A^{(r)} \leftarrow A^{(r)} \cup \{x\}$ .
27:  end while
28:  Oracle verification/editing: for each  $x \in A^{(r)}$ , oracle provides true  $y$  and edits  $\hat{e} \mapsto \tilde{c}$ .
29:   $\mathcal{A}_e^{(r)} \leftarrow \{(x, \tilde{c}) : x \in A^{(r)}\}$ .
30:  Monitoring:  $\text{acc}^{(r)} = \frac{1}{|A^{(r)}|} \sum_{x \in A^{(r)}} \mathbf{1}[\hat{y}(x) = y(x)]$ .
31:  Update prompt set:  $\mathcal{P}^{(r)} \leftarrow \mathcal{P}^{(r-1)} \cup \mathcal{A}_e^{(r)}$ .
32:  Update pool:  $\mathcal{D}_u \leftarrow \mathcal{D}_u \setminus A^{(r)}$ .
33:  if stopping rule holds (APT) then
34:     $R \leftarrow r$ ;
35:    break.
36:  end if
37: end for
38: return  $\mathcal{P}^{(R)}$ .
```

References

- [1] Siddharth Karamcheti, Ranjay Krishna, Li Fei-Fei, and Christopher Manning. Mind your outliers! investigating the negative impact of outliers on active learning for visual question answering. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7265–7281, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.564. URL <https://aclanthology.org/2021.acl-long.564/>.
- [2] B. W. Silverman. *Density Estimation for Statistics and Data Analysis*. Chapman & Hall/CRC, 1 edition, 1998. doi: 10.1201/9781315140919.